《信息安全导论》实验报告
实验名称：简化 AES(S-AES)综合加解密工具的设计与实现
小组成员：张瑞淇，何润华，樊帆
实验日期：2026 年 10 月 18 日
GitHub 链接：https://github.com/wen-yan-chen/pro_2_SAES

一、实验目的与要求

本实验旨在通过编写程序，深入理解分组密码算法（S-AES）的原理及工作模式。具体包含以下 5 个关卡的要求：

1. 第 1 关：实现 16bit 明文与 16bit 密钥的标准 S-AES 加密与解密。
2. 第 2 关：与其他小组进行交叉测试，验证双方算法实现的一致性。
3. 第 3 关：扩展功能，支持 ASCII 字符串的分组加解密
4. 第 4 关：实现双重加密、三重加密，并编程实现中间相遇攻击（Meet-in-the-Middle Attack）。
5. 第 5 关：实现密码分组链（CBC）工作模式，并验证密文篡改后的错误传播现象。

二、实验环境

- 操作系统：Windows11/macOS/Linux
- 开发语言：Python 3.8+（实测 Python 3.13）
- 依赖库：仅使用 Python 标准库（tkinter 等），无第三方依赖
- 开发工具：PyCharm

三、系统设计与接口封装(对应 5.2.3 开发手册)

项目采用算法层与界面层分离的设计原则，算法层（saes/ 目录）不依赖任何界面代码，可直接被其他程序复用。

1) 核心模块划分

saes/core.py：核心算法，包含 S 盒、密钥扩展、单分组加解密。
saes/multi_crypto.py：多重加密，包含双重、三重加密以及中间相遇攻击。
saes/cbc_mode.py：工作模式，实现 CBC 加解密。
gui/app.py：图形界面层，仅负责输入解析和结果展示。

2) 核心接口说明(API 文档)

为方便二次开发，组件已封装为纯函数，主要接口如下：

模块	函数签名	说明
单次加密	s_aes_encrypt(plaintext: int, key: int)->int	输入 16bit 明文和密钥，返回 16bit 密文。
单次解密	s_aes_decrypt(ciphertext: int, key: int) -> int	输入 16bit 密文和密钥，返回 16bit 明文。
双重加密	double_encrypt(plaintext: int, key32: int) -> int	输入 16bit 明文和 32bit 密钥（K1+K2），返回密文。
三重加密	triple_encrypt(plaintext: int, key48: int) -> int	输入 16bit 明文和 48bit 密钥，采用 EDE 结构加密。
中间相遇	meet_in_the_middle_attack(...) -> list	输入已知明密文对，返回恢复的[(K1,K2),...]候选列表。
CBC 加密	cbc_encrypt(plaintext_bytes: bytes, key16: int, iv16: int) -> list[int]	将字节串分组，返回密文分组列表。

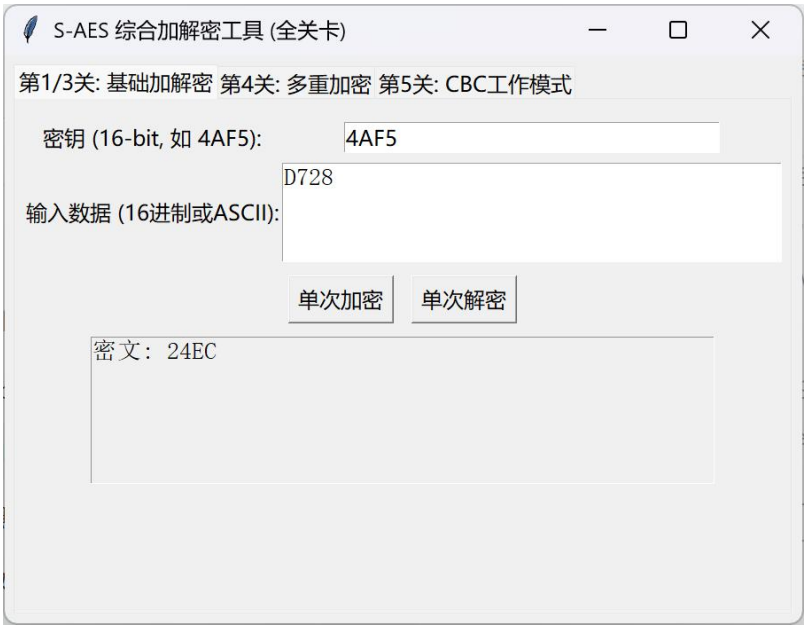
四、实验测试结果(对应 5.2.2 测试结果)

根据 5 个关卡的要求，测试结果如下（详细截图已存放于 GitHub 仓库 docs/screenshots/目录）。

4.1 第 1 关：基本测试

- 测试目的：验证 16bit 数据与密钥的加解密流程。
- 测试数据：密钥 K=4AF5，明文=D728。
- 测试结果：
  - 加密输出：24EC（与教材标准向量一致）
  - 解密输出：D728（成功还原）

第 1 关-加密结果截图：



第 1 关-解密结果截图：



4.2 第 2 关：交叉测试

测试目的：验证不同实现（本组 A 端与另一小组 B 端）在同一标准下的互通性。

测试结果：

双方使用 5 组固定明文及 200 组随机抽样进行比对，加密结果完全一致，且互相解密成功。证明本组算法流程与 S 盒、列混淆矩阵的选取完全符合标准。

类别	明文(P)	密钥(K)	A 端密文	B 端密文	结论
教材向量	D728	4AF5	24EC	24EC	一致
交叉测试	0000	4AF5	52B1	52B1	一致
交叉测试	FFFF	4AF5	74DB	74DB	一致
随机抽样	200 组随机数	-	-	-	200/200 一致

第 2 关-交叉测试运行截图：



4.3 第 3 关：扩展功能（ASCII 字符串）

- 测试目的：支持 ASCII 字符串输入并按 2 字节（16bit）分组处理。
- 测试数据：密钥 4AF5，明文 HELLO（5 字节，奇数长度末尾补 1 个 0x00，共 3 个分组）。
- 测试结果：
- 加密输出：08D7 4BF1 1FE0（分组对应：HE→ 08D7，LL → 4BF1，O+补零 → 1FE0）
- 解密输出：HELLO（自动过滤补齐的空字符）

第 3 关-ASCII 加密截图：



第 3 关-ASCII 解密截图：



4.4 第 4 关：多重加密

4.4.1 双重加密

- 测试数据：密钥 4AF51234(K1=4AF5,K2=1234)，明文 D728。
- 测试结果：加密输出 4A90，解密可还原。

第 4 关-双重加密截图：



第 4 关-双重解密截图：



4.4.2 三重加密

- 测试数据：密钥 4AF51234ABCD，明文 D728。
- 测试结果：加密输出 68A6，解密可还原。

第 4 关-三重加密截图：



第 4 关-三重解密截图：



4.4.3 中间相遇攻击(关键分析)

- 攻击原理：双重加密密钥空间为 232232。通过枚举 K1K1 计算中间值建立字典，再枚举 K2K2 进行反向碰撞，将复杂度降至 2×2162×216。
- 测试数据：已知明密文对：1(P=D728,C=4A90P=D728,C=4A90)，明密文对 2：(P=1234,C=E3B5P=1234,C=E3B5)。
- 测试结果：耗时约 5.6 秒，成功恢复唯一密钥 K1=4AF5,K2=1234。
- 假阳性分析：由于分组仅 16bit，单对明密文会产生约 6.5 万个碰撞候选。本程序引入了第二对明密文进行复核，成功将候选收敛至 1 个，证明了攻击的有效性。

第 4 关-中间相遇攻击运行截图：



4.5 第 5 关：工作模式（CBC）与密文篡改

- 测试数据：密钥 4AF5，IV=0000，明文 HELLO CBC MODE（14 字节，7 个分组）。
- 正常加解密：
  - 密文：08D7 6316 F6C6 A1C7 394A 5160 7D16
  - 解密：HELLO CBC MODE

第 5 关-CBC 加密截图：



第 5 关-CBC 解密截图：



篡改测试与错误传播分析：

将第 2 个密文块 6316 修改为 6317（仅翻转最后 1bit），再次解密。

- 解密结果：HE<NO!CBC MODE
- 现象分析：
  - 第 1 块(HE)：未受影响。
  - 第 2 块(LL)：解密后完全变为乱码(<N)。
  - 第 3 块(O)：对应比特发生翻转（空格 0x20 变为!0x21）。
  - 后续块：未受影响，正常输出。
- 结论：验证了 CBC 模式特有的错误传播特性（篡改影响自身及下一块），同时证明仅靠 CBC 无法发现篡改，实际系统需引入 MAC/HMAC 保证完整性。

第 5 关-CBC 密文篡改运行截图：



五、总结与反思

1. 算法实现的严谨性：在实现 S-AES 时，状态矩阵的“列优先”原则是易错点。我们编写了完整的自检脚本确保了算法的正确。
2. 安全性与分组长度的关系：通过中间相遇攻击实验，我们深刻认识到 16bit 分组极易引发碰撞（假阳性极高），证明了 S-AES 仅适用于教学。
3. 模式与完整性的剥离：CBC 篡改实验让我们直观地看到了“加密”与“完整性保护”的边界。在生产环境中，必须结合 HMAC 或使用 AEAD 模式。
4. 工程化思维：本次实验将算法内核与 GUI 界面完全解耦，实现了纯函数封装，不仅方便单元测试，也使得算法可以无缝接入命令行或 Web 后端。